

1. CGってなに？ 3Dゲーム画面のしくみ



CGは、コンピューターが
物体・光・カメラ を計算して、
2D画面に映し出すしくみです。

1 ライト（光）



明るさと影を作る

ライトがあると、明るいところと
暗いところができ、立体感が
生まれます。

2 オブジェクト（物体）

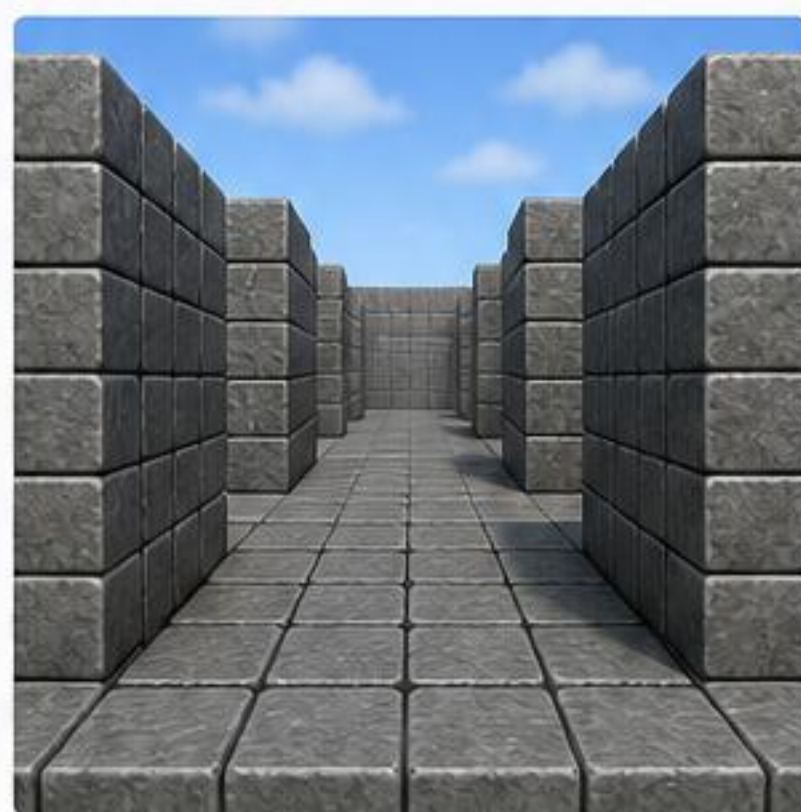
 床ブロック・壁ブロック・
旗・プレイヤー



ゲームの中にあるすべてのものが
「オブジェクト（物体）」です。
形や見た目を決めます。

3 カメラ（視点）

 どこから見るかを定める



カメラが、プレイヤーの「目」に
なります。カメラの位置や向きで、
見える画面が変わります。

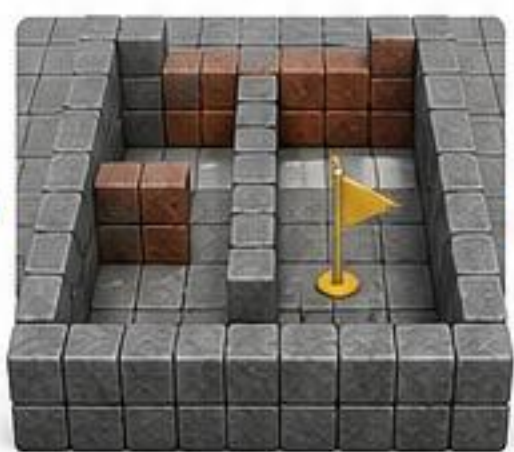
この迷路サンプルでは

1 光で照らす



ライトで迷路を照らして、
明るさと影を作る。

2 立方体ブロックを並べる



床ブロックと壁ブロックを
立方体で並べて迷路を作る。

3 カメラで見る



カメラ（視点）で迷路の中を
見ることができる。

4 プレイヤーが進む



プレイヤーを操作して、
迷路の中を進む。

5 旗でゴール



旗のあるマスに入ると
ゴールになる！



1マスは床ブロック単位で作る



=

1マス（床ブロック）



ライト・オブジェクト・カメラの3つを組み合わせることで、
3Dゲームの世界を作ることができます！

2.

ノードをまとめてシーンにする

Godotでは、部品をノードで作り、親子関係でまとめて、ひとつのシーンとして再利用します。

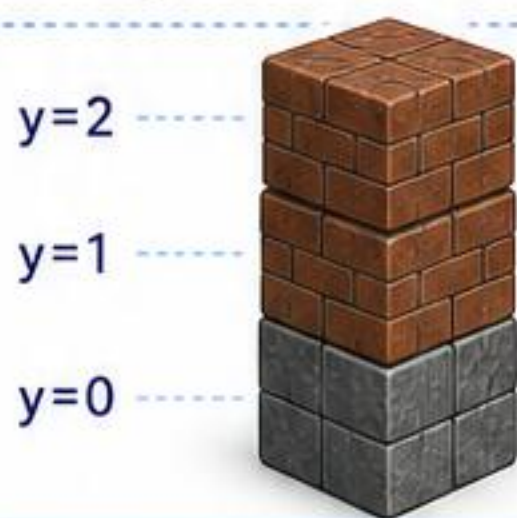
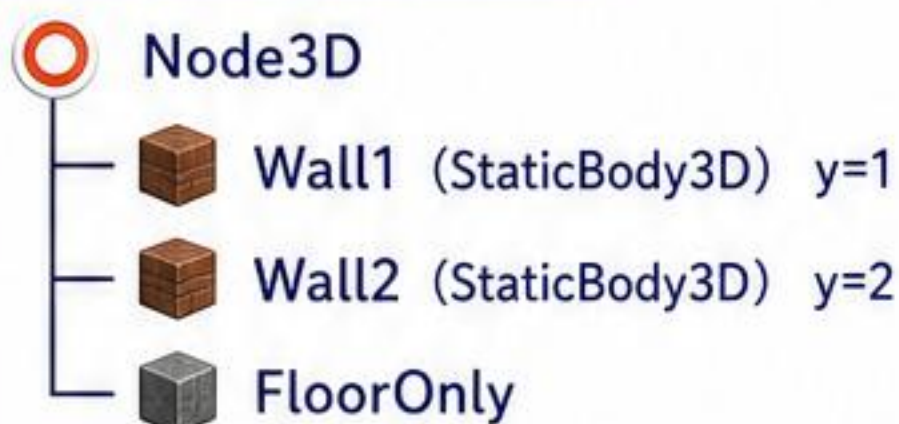
シーンの考え方

FloorOnly.tscn



床ブロック
1マス

FloorAndWall.tscn



床+壁2段
(高さ3マス)

主なノードと必要な理由



Node3D

3D部品をまとめる



MeshInstance3D

ブロックや旗を見せる



StaticBody3D

動かない床・壁にする



CollisionShape3D

すり抜けを防ぐ



CharacterBody3D

プレイヤーを動かす



Camera3D

FPS視点にする



Area3D

ゴールに入ったことを検知



CanvasLayer / Label

GOAL!を画面に出す

Main.tscnでは

Mazeノードの下に、FloorOnly / FloorAndWall / FloorAndGoal / FloorAndPlayer を並べて5×5の迷路にします。



FloorOnly



FloorAndWall



FloorAndGoal



FloorAndPlayer



5×5の迷路イメージ



シーンを分けてまとめることで、作りやすく・使い回しやすく・管理しやすいゲームになります!



3.

ブロックを並べて迷路を作る

このサンプルでは、1マス进行床ブロック単位で作り、MainシーンのMazeノードに5×5で並べています。

使うブロックシーン

1 FloorOnly

石の床ブロック 1個



BoxMesh + BoxShape3D

2 FloorAndWall

床ブロック + 壁ブロック2段



Wall2
y=2
Wall1
y=1
Floor
y=0

BoxMesh + BoxShape3D (各ブロック)

3 FloorAndGoal

床ブロック + 旗のArea3D



BoxMesh + BoxShape3D
+ Area3D + CollisionShape3D

main.tscnの配置

5×5のグリッドに、左のブロックを並べます。

	0	1	2	3	4
0	壁	壁	壁	壁	壁
1	壁	床	開始	床	壁
2	壁	壁	壁	床	壁
3	壁	ゴール	床	床	壁
4	壁	壁	壁	壁	壁



開始 = FloorAndPlayer



ゴール = FloorAndGoal

実コードとの対応

1 floor_only.tscn: BoxMeshの床ブロック



1マスの床ブロックです。

2 floor_and_wall.tscn: Wall1(y=1) と Wall2(y=2)



Wall2
y=2
Wall1
y=1
Floor
y=0

床の上に壁を2段重ねたブロックです。

3 main.tscn: Mazeの下に各マスを配置



5×5の迷路をブロックで作ります。

4.



Input MapとFPS視点を作る

Input Mapは、キー入力に名前をつける場所です。
コードではキー名ではなく、操作名を使います。



project.godotのInput Map

Project Settings → Input Map に、
このように登録されています。

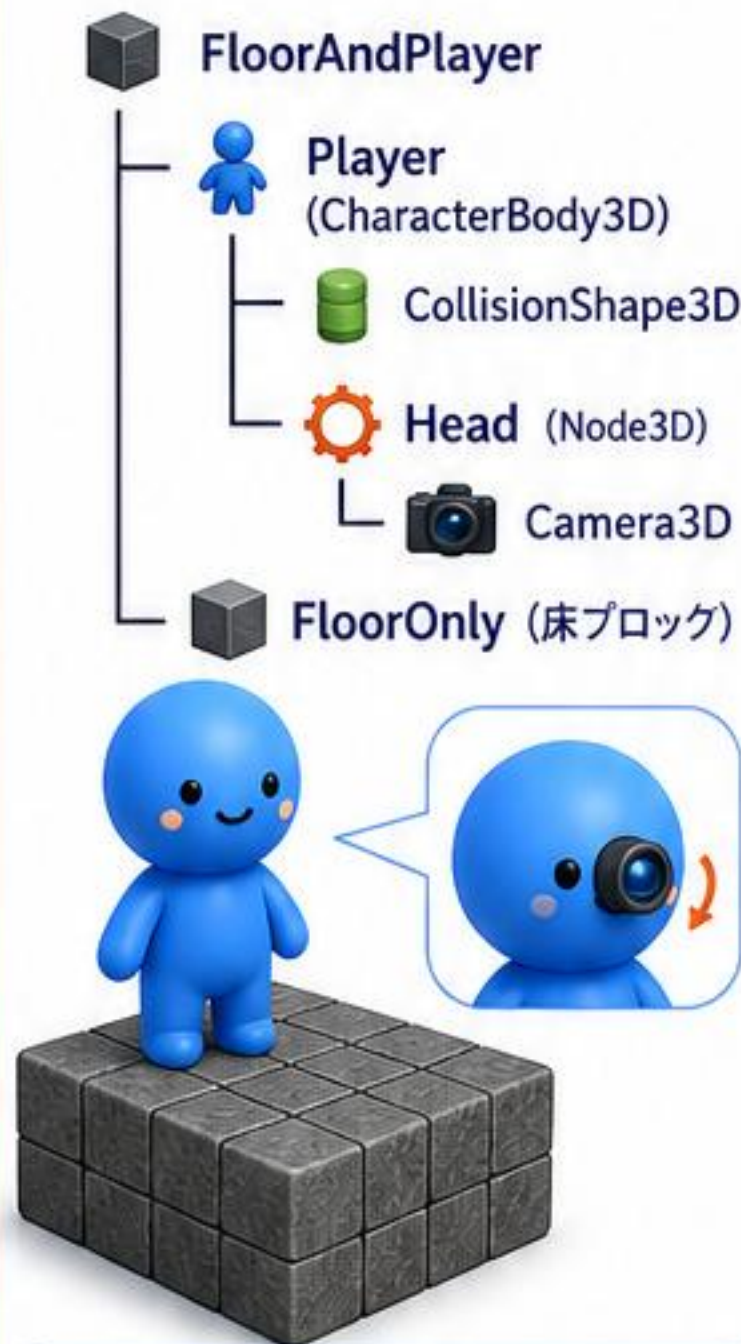
操作名	キー	動き
move_forward	W	前へ進む
move_backward	S	後ろへ下がる
move_left	A	左へ移動
move_right	D	右へ移動
ui_accept	Space	ジャンプ



操作名を使うことで、あとからキーを
変更してもコードを直さなくて済みます！

プレイヤーのノード構成

プレイヤーと床の関係を、
親子ノードで組み立てます。



カメラは Head の中にあるので、
プレイヤーの「目」として見えます。

player.gdの役割

player.gdは、プレイヤーの動きや
視点のしくみを担当します。



SPEED = 5.0
歩く速さ



JUMP_VELOCITY = 4.5
ジャンプの強さ



MOUSE_SENSITIVITY = 0.002
視点操作の感度



Input.get_vector
WASDを移動方向にする



move_and_slide
当たり判定つきで動く

FPS視点の回転

マウスの動きで、Player（体）と Head（頭）をそれぞれ回転させて、FPS視点を作ります。

マウス左右 → Playerを横回転

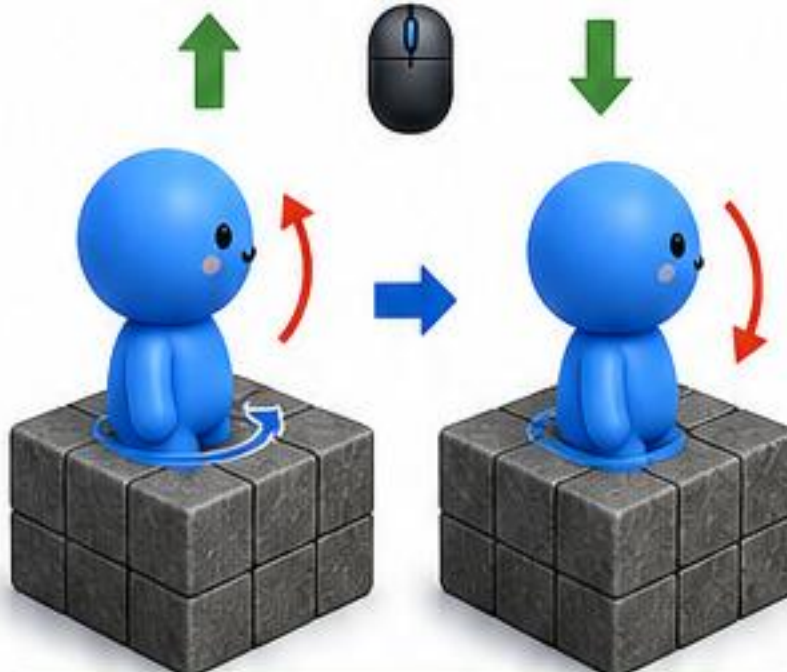
マウスを左右に動かすと、プレイヤーの体全体が
水平方向に回転します。



体を回すことで、見ている方向（進む向き）を
変えられます。

マウス上下 → Headを上下回転

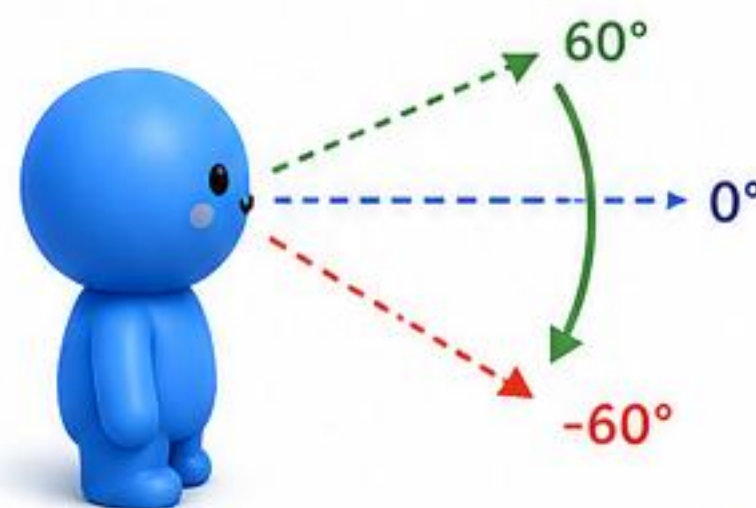
マウスを上下に動かすと、Head（頭）だけが
上下に回転します。



体はそのままで、見上げたり見下ろしたり
できます。

上下角度は -60°～60°

見上げすぎ・見下ろしすぎを防ぐために、
上下の角度を制限しています。



-60°より下、60°より上には
回転しません。



Escでマウスを表示、クリックで再キャプチャ

ゲーム中に Esc キーでマウスカーソルを表示できます。
また、画面をクリックすると、マウスを再キャプチャして操作を再開します。





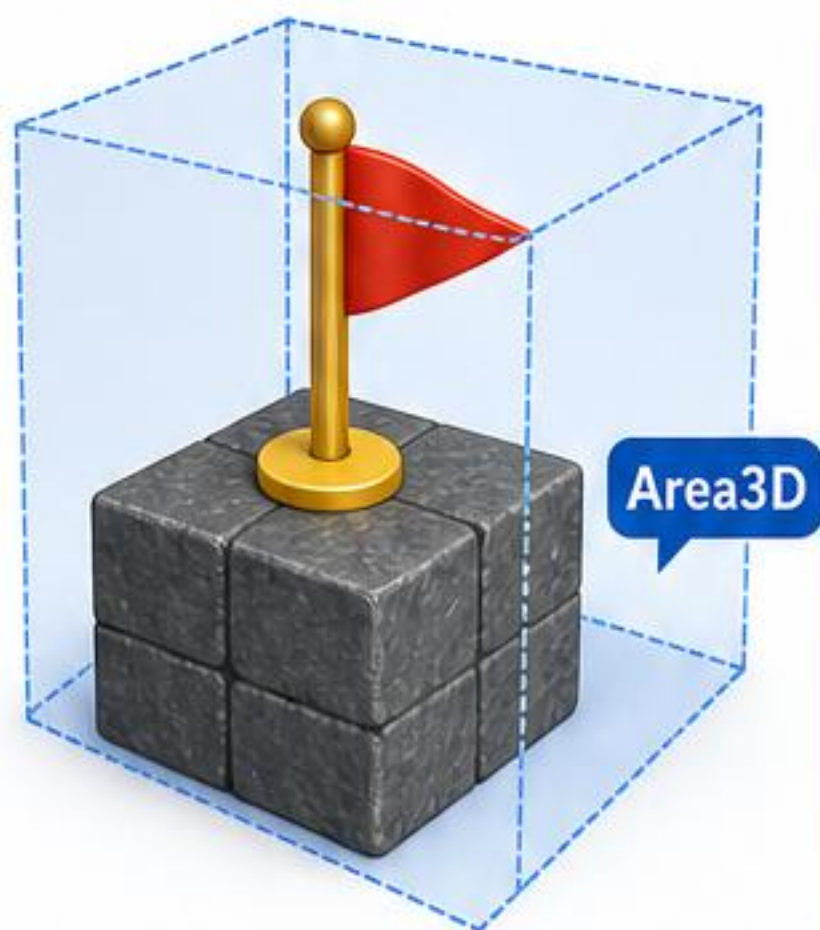
5.

ゴール判定とHUDを作る

ゴールは「ぶつかる壁」ではなく、
「入ったことを知らせるエリア」として作ります。



FloorAndGoalの構成



処理の流れ

1 プレイヤーが旗のエリアに入る



2 body_entered シグナル



3 HUDに表示をお願いする



4 GoalLabelが表示される



HUDの構成

HUD (CanvasLayer)
group: hud

A GoalLabel (Label)
visible = false から開始

画面表示の例



ポイント

CanvasLayerは3D空間とは別に、画面の上に文字を表示します。

実コードで確認する場所

scripts/flag.gd



旗に入ったら
HUDを探して呼び出す

scripts/hud.gd



GoalLabelを表示する

scenes/main.tscn



HUDとGoalLabelを持つ



判定は **Area3D**、画面表示は **CanvasLayer**。役割を分けるとわかりやすい。





6. 作る順番：サンプル迷路の完成まで

いきなり全部作らず、動くところまで小さく確認しながら進めます。



1



プロジェクトを作る
main.tscnを
起動シーンにする



2



床ブロックを作る
FloorOnly: BoxMesh
+ BoxShape3D



3



壁ブロックを積む
Wall1(y=1) と
Wall2(y=2)



4



Mazeに5×5で並べる
床・壁・開始・ゴールを配置



5



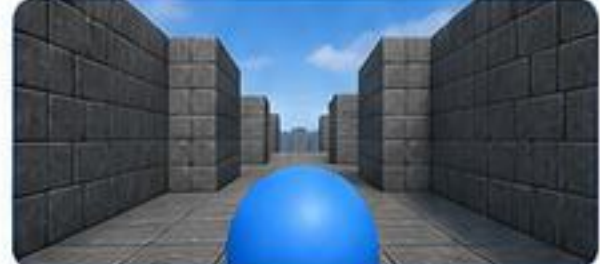
プレイヤーを作る
CharacterBody3D +
Head + Camera3D



6



Input Mapとplayer.gd
WASD移動・マウス視点・
ジャンプ



7



ゴールとHUD
Area3Dで判定し、
GOAL!を表示



完成チェック



床が立方体ブロックで
並んでいる



壁は床ブロックの上に
2段積み



WASDで
移動できる



マウスで
視点が動く



壁を
すり抜けない



旗に入ると
GOAL!が出る



次に足せるアイデア

音

AudioStreamPlayer3D



歩く音やゴール音などで
ゲームをもっと楽しく！

視線判定

RayCast3D (目から線)



壁やオブジェクトを
見ているかを判定！

ステージ選択



複数のステージを作って
好きなステージを選ぼう！

タイマー



クリア時間を計測して
ランキングにしよう！

アイテム



コインや鍵などを集めて
仕掛けを開けよう！



対応ファイル： project.godot / scenes/main.tscn / scenes/floor_only.tscn /
scenes/floor_and_wall.tscn / scripts/player.gd / scripts/flag.gd / scripts/hud.gd

